Лабораторная работа № 4

Взаимодействие со стандартными приложениями Windows на VBScript/JScript

Дисциплина: «Операционные системы, среды и оболочки».

Студентка: _____

Группа: ИИ-551. Преподаватель: _____

Дата выполнения и сдачи: _____

О документе

Отчёт подготовлен на основе предоставленного файла «ОС4.docx». Исходные 12 скриншотов сохранены и сгруппированы по заданиям. К ним добавлены условия, последовательность действий, объяснения команд и алгоритмов. Код script3.js перенесён из исходного Word-документа; листинги первых двух заданий соответствуют версии со вспомогательными процедурами CalcKeys и NoteKeys, показанной на скриншотах. Имена, дата сдачи и преподаватель не придуманы: соответствующие поля оставлены для заполнения.

Важно различать версию на скриншотах и последующие исправления. На снимках первых двух заданий показан ввод через SendKeys; более поздняя версия файлов записывает подписи в Unicode-файл. Описание результатов ниже относится к фактически приложенным снимкам, а не к неподтверждённому повторному запуску исправленных программ.

1. Цель и задачи работы

Цель: изучить программный запуск приложений Windows, переключение между их окнами, передачу клавиатурных команд, файловый ввод-вывод и диалоговый выбор параметров сценария.

Задачи: выполнить 9 × 6 в Калькуляторе и перенести ответ в Блокнот; сложить F7 и 7F в шестнадцатеричной системе, получить HEX- и DEC-представления и вернуть обычный вид Калькулятора; обработать файл выражений с четырьмя системами счисления, дробями и скобками, записав ответы в выбранном основании.

2. Среда, файлы и границы подтверждения

Рабочая система пользователя — Windows 11. На снимках запущен классический интерфейс Калькулятора и Блокнот. Точная сборка Windows и версия Калькулятора на снимках не указаны. Облачная среда, в которой готовились пояснения, работает под Linux; новые запуски окон Windows при подготовке этого отчёта не выполнялись.

Рабочий каталог по данным переписки: C:\STUDENT\ИИ-551\Lab4. Имя группы при необходимости заменяется фактическим. Все связанные файлы следует переносить вместе.

Файл	Назначение
script1.vbs	Первый сценарий VBScript: 9 × 6, вывод в Блокнот.
script2.vbs	Второй сценарий: HEX-сумма, DEC-представление, возврат обычного режима.
script3.js	JScript: диалоги, чтение выражений, разбор, запросы Калькулятору, запись результатов.
calculator.ps1	Вспомогательный PowerShell-модуль третьего сценария; в поздней версии используется всеми тремя заданиями.
text1.txt	Вход: одно выражение в строке.
text2.txt	Выход третьего задания; при новом запуске перезаписывается.
run.cmd	Меню выбора сценария для Windows Script Host.

Само наличие исходного кода подтверждает способ реализации, но не доказывает успешный запуск. Скриншоты показывают конкретное состояние окон и файлов. Поддержка всех ветвей программы требует отдельных проверок; её нельзя выводить только из одного успешного примера.

3. Основные понятия

Windows Script Host (WSH) — среда выполнения сценариев Windows. wscript.exe показывает сообщения WScript.Echo в окнах, cscript.exe выводит их в консоль. Файл .vbs выполняется движком VBScript, .js в этой работе — движком JScript WSH, а не браузером и не Node.js.

WScript — объект среды выполнения: через него доступны Sleep, Echo, Quit, ScriptFullName. WScript.Shell — отдельный COM-объект, создаваемый CreateObject; он предоставляет Run, Exec, AppActivate, SendKeys и Popup. Scripting.FileSystemObject — ещё один COM-объект для работы с файлами и каталогами.

Процесс — выполняющийся экземпляр программы. Окно — элемент её интерфейса. PID идентифицирует процесс, HWND — окно; это разные сущности. AppActivate принимает PID или заголовок окна. Он не является общим методом перечисления всех процессов. Приложение может создать несколько окон, а при запуске современного приложения процесс-лаунчер может передать управление другому процессу.

Буфер обмена — общий ресурс сеанса Windows. Ctrl+C просит активное приложение скопировать данные, Ctrl+V вставляет их в активное приложение. Наличие на дисплее правильного числа не гарантирует, что оно было скопировано или вставлено.

4. Подготовка каталога и запуск

Команды ниже выполняются в cmd.exe. Если папка уже создана, повторное создание не требуется.

```
mkdir "C:\STUDENT\ИИ-551\Lab4"
cd /d "C:\STUDENT\ИИ-551\Lab4"
dir
notepad text1.txt
cscript.exe //nologo script1.vbs
cscript.exe //nologo script2.vbs
cscript.exe //nologo //E:JScript script3.js
```

Команда / элемент	Объяснение
mkdir "путь"	Создаёт каталог. Кавычки позволяют использовать пробелы в пути.
cd /d "путь"	Меняет текущий каталог и, благодаря /d, текущий диск.
dir	Показывает содержимое каталога; позволяет проверить наличие зависимостей.
notepad text1.txt	Открывает входной файл для редактирования.
cscript.exe	Консольный хост сценариев Windows.
//nologo	Убирает стартовый баннер WSH; на вычисления не влияет.
//E:JScript	Явно выбирает JScript, даже если ассоциация .js изменена редактором.
wscript.exe script1.vbs	Альтернативный оконный запуск; двойной щелчок обычно использует его.

Команды запускаются по одной с ожиданием завершения. Перед автоматизацией закрываются лишние окна Калькулятора и Блокнота, выбирается английская раскладка. Во время передачи клавиш нельзя переключать фокус. Значения из буфера обмена, которые нужно сохранить, копируются заранее в другой документ.

Файлы .vbs сохраняются как Unicode UTF-16LE. Вход text1.txt содержит только ASCII-символы выражений; для данной реализации подходит ANSI или UTF-8 без BOM. UTF-16-вход потребовал бы другого режима OpenTextFile. Выход text2.txt создаётся в UTF-16LE.

5. Задание 1: вычисление 9 × 6

5.1. Последовательность действий

Сценарий создаёт WScript.Shell, предупреждает пользователя, запускает calc.exe и notepad.exe с двухсекундными паузами. Затем выбирает обычный режим Калькулятора, очищает предыдущее состояние и отправляет 9*6=. После Ctrl+C активируется Блокнот, вводится подпись и выполняется вставка Ctrl+V. В конце отправляется Enter.

```
CalcKeys "%1"
CalcKeys "{ESC}"
CalcKeys "9*6="
CalcKeys "^c"
NoteKeys "Результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"
```

CalcKeys и NoteKeys — пользовательские процедуры, а не встроенные команды Windows. Они сначала вызывают процедуру активации нужного окна, затем SendKeys и задержку 700 мс. Такой порядок уменьшает вероятность отправки клавиш до готовности интерфейса, но фиксированная задержка не гарантирует успех при любой нагрузке.

5.2. Объяснение конструкций VBScript

Конструкция	Что делает и зачем используется
Option Explicit	Требует объявления переменных; помогает обнаруживать опечатки в именах.
Dim sh	Объявляет переменную для ссылки на объект оболочки.
Set sh = CreateObject("WScript.Shell")	Создаёт COM-объект и присваивает объектную ссылку. Set нужен для объектов в VBScript.
sh.Run "calc.exe", 1, False	Запускает Калькулятор; 1 — обычное активное окно, False — не ждать закрытия программы.
sh.Run "notepad.exe", 1, False	Аналогично запускает Блокнот.
WScript.Sleep 2000	Приостанавливает сценарий на 2000 мс, то есть на две секунды.
sh.AppActivate("Калькулятор")	Пытается активировать окно по заголовку; возвращает True или False.
For i = 1 To 30	До 30 попыток найти окно, между попытками выполняется пауза.
If ... Then / End If	Выполняет действия при истинном условии.
Exit Sub	Завершает текущую процедуру после успешной активации.
MsgBox ..., 16, ...	Выводит сообщение со значком ошибки.
WScript.Quit 1	Завершает весь сценарий с ненулевым кодом ошибки.
Sub CalcKeys(keys)	Объявляет процедуру с параметром — строкой клавиатурных команд.
sh.SendKeys keys	Посылает указанную последовательность клавиш активному окну.

В условии исходной версии используется Or между попытками активации русскоязычного и англоязычного окна. VBScript не делает короткого замыкания Or: обе части могут вычисляться. При нескольких подходящих окнах это создаёт неоднозначность; для надёжного выбора предпочтительны последовательные проверки либо известный PID.

5.3. Наблюдение и вывод

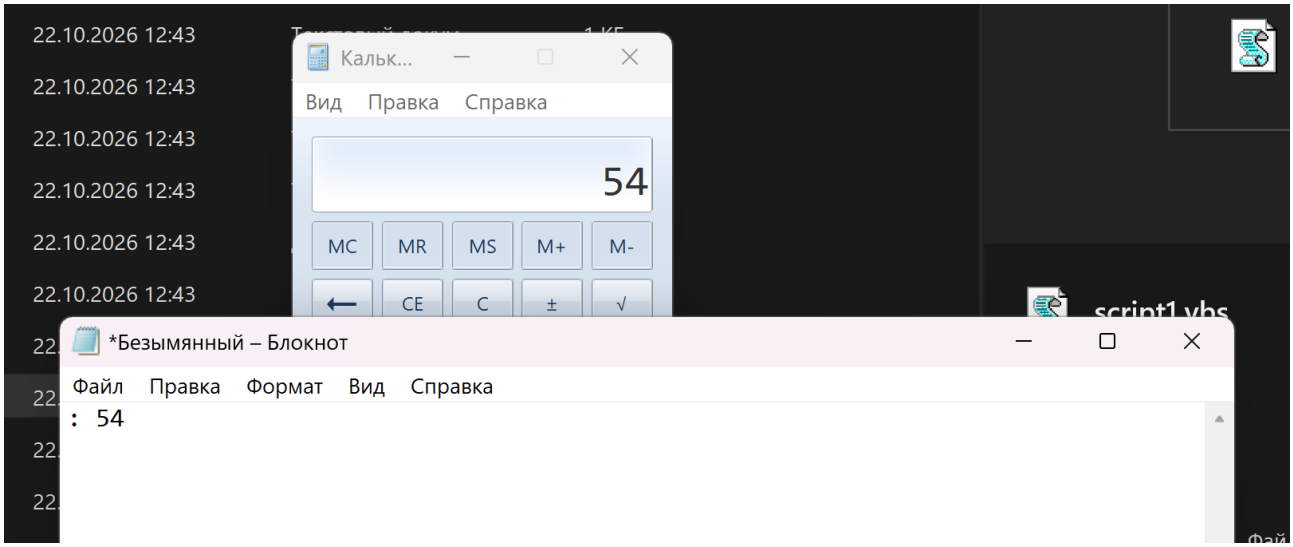


Рисунок 1 — Результат первого задания: на дисплее Калькулятора 54, в Блокноте виден перенесённый ответ 54; полная русская подпись отсутствует.

Арифметическая проверка: 9 × 6 = 54. Снимок подтверждает числовой результат и его наличие в Блокноте. Он также показывает дефект отображения подписи: вместо полной строки «Результат: 54» видны двоеточие и число. Поэтому оформление нельзя объявлять полностью корректным по этому изображению.

В последующей версии подпись и считанный ответ записываются в текстовый файл через CreateTextFile(..., True, True), затем файл открывается Блокнотом. Это исключает имитацию ввода кириллицы. Скриншот повторного запуска этой исправленной версии в исходный документ не включён.

6. Задание 2: сложение F7 и 7F

6.1. Последовательность действий

После запуска приложений выбираются режим «Программист» (Alt+3) и основание HEX (F5). После очистки вводится F7{+}7F=. HEX-ответ копируется в Блокнот. Затем активируется Калькулятор, выбирается DEC (F6), копируется десятичное представление того же результата. Завершающая команда Alt+1 возвращает обычный режим.

```
CalcKeys "%3"
CalcKeys "{F5}"
CalcKeys "{ESC}"
CalcKeys "F7{+}7F="
CalcKeys "^c"
NoteKeys "Шестнадцатеричный результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"
CalcKeys "{F6}"
CalcKeys "^c"
NoteKeys "Десятичный результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"
CalcKeys "%1"
```

Запись SendKeys	Значение
%3	Alt+3 — выбор режима «Программист» в используемой реализации Калькулятора.
{F5}	Нажатие F5: выбор HEX в режиме программиста.
F7	Два обычных символа: цифра F и цифра 7, то есть число $F7_{16}$.
{F7}	Функциональная клавиша F7. Это другая команда; в программистском режиме выбирает OCT.
{+}	Буквальный знак сложения. Без скобок + обозначает модификатор Shift.
{F6}	Нажатие F6: выбор DEC.
%1	Alt+1 — возврат обычного режима.

6.2. Проверка числа

$F7_{16} = 15 \times 16 + 7 = 247_{10}$. $7F_{16} = 7 \times 16 + 15 = 127_{10}$. Сумма равна 374_{10}. Деление на 16 даёт остатки 6, 7, 1; при чтении в обратном порядке получается 176_{16}.

Ожидаемый формат вывода: «Шестнадцатеричный результат: 176», затем «Десятичный результат: 374». Основание меняет запись числа, а не само значение.

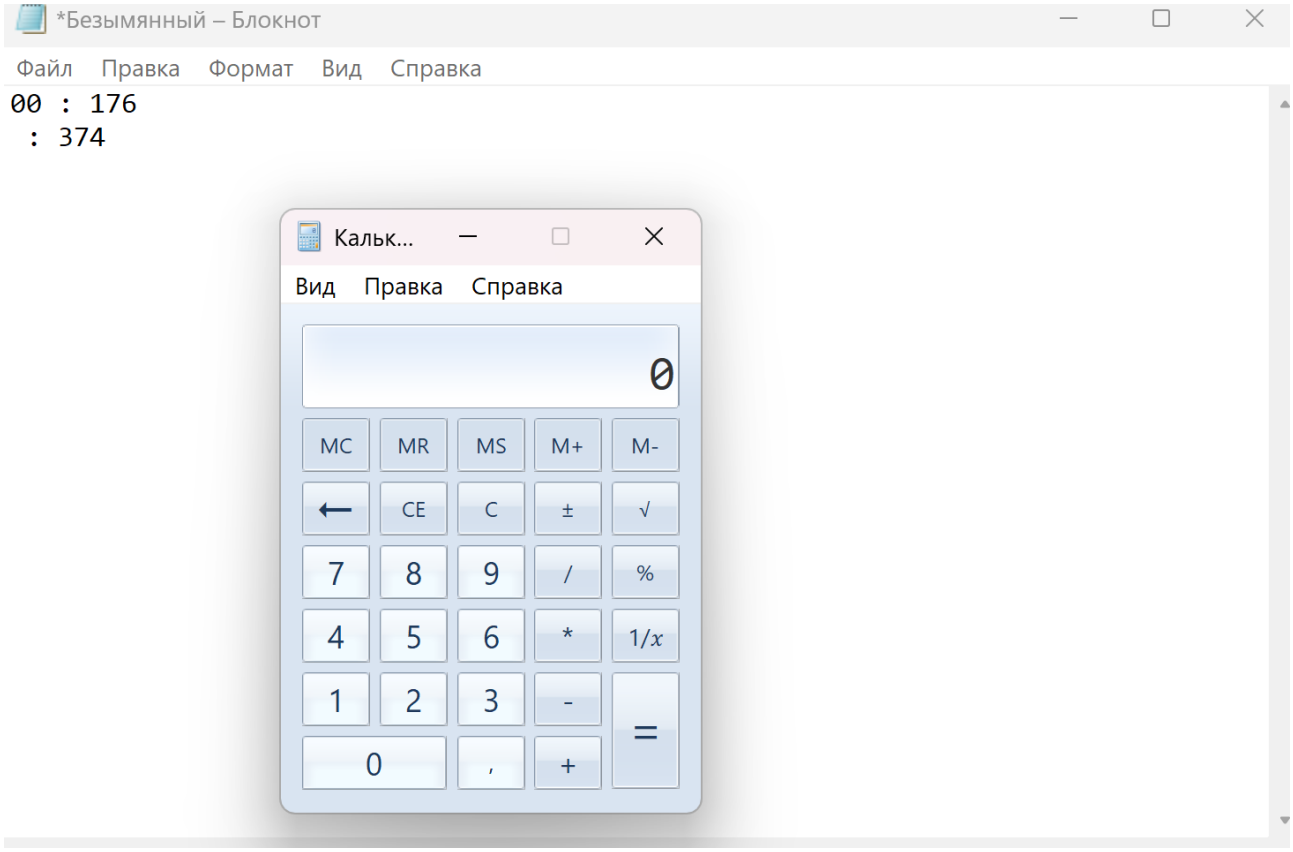


Рисунок 2 — На снимке второго задания видны 176 и 374; Калькулятор отображён в обычном режиме. Русские подписи искажены.

Снимок подтверждает два ожидаемых числовых значения и обычный вид окна в момент снимка. Однако поясняющие слова не отображены в требуемом виде. Исправление записи подписи в Unicode-файл описано в разделе 5.3; факт повторного успешного оформления по этому снимку не устанавливается.

7. Задание 3: обработка выражений из файла

7.1. Входные данные и общий маршрут

Для основного запуска использованы шесть примеров:

```
9*6
0xF7+0x7F
0x1A+2
@101+1
(2+3)*4
0x1.A+0x2.8
```

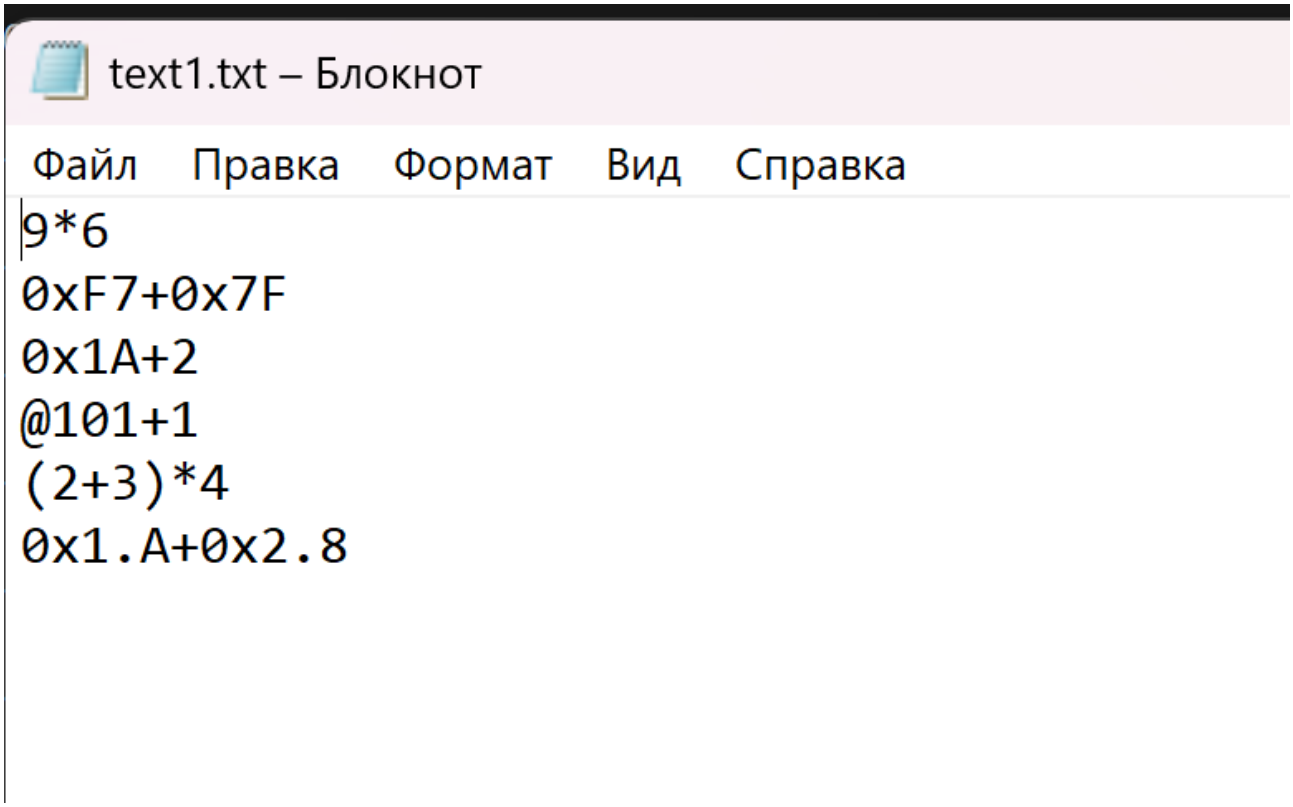


Рисунок 3 — Входной файл text1.txt с шестью выражениями.

Маршрут данных: text1.txt → ReadLine → parse → дерево выражения → evaluate → bridge → calculator.ps1 → Калькулятор → число-ответ → format → text2.txt. Если в выражении несколько операций, Калькулятор вызывается по одному разу для каждого бинарного узла дерева. Для одиночного числа используется запрос identity.

JScript отвечает за синтаксис, перевод оснований и выбор порядка операций. Калькулятор выполняет бинарные +, −, ×, ÷. PowerShell обеспечивает связь с окном и получение ответа. Поэтому ошибочно говорить, что Калькулятор самостоятельно разбирает строку с префиксами разных оснований и скобками.

7.2. Запуск и выбор основания результата

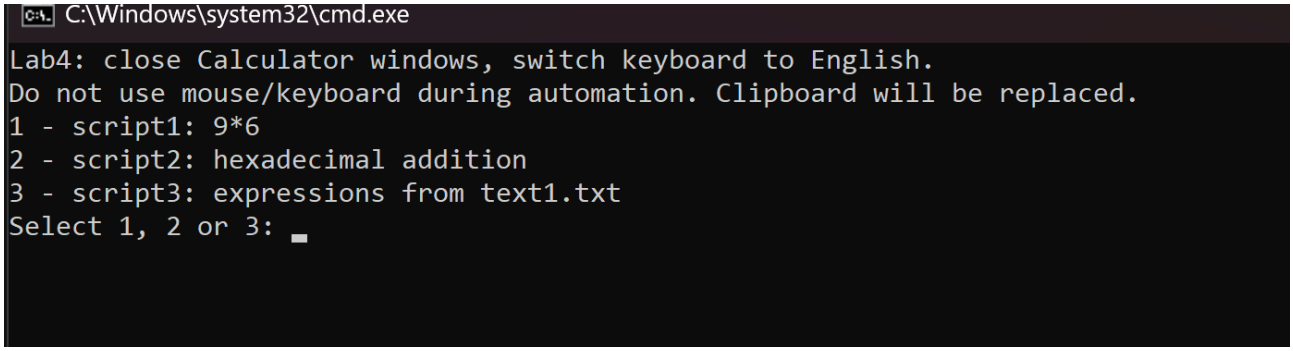


Рисунок 4 — Меню run.cmd: для третьего задания выбирается пункт 3.

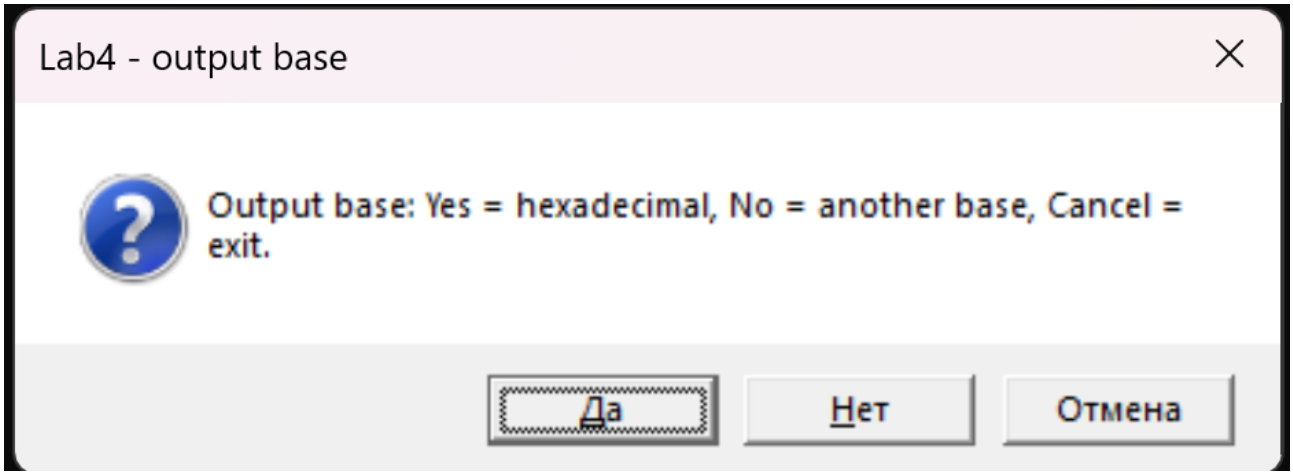


Рисунок 5 — Первый Popup: «Да» выбирает HEX, «Нет» открывает следующий диалог, «Отмена» завершает сценарий.

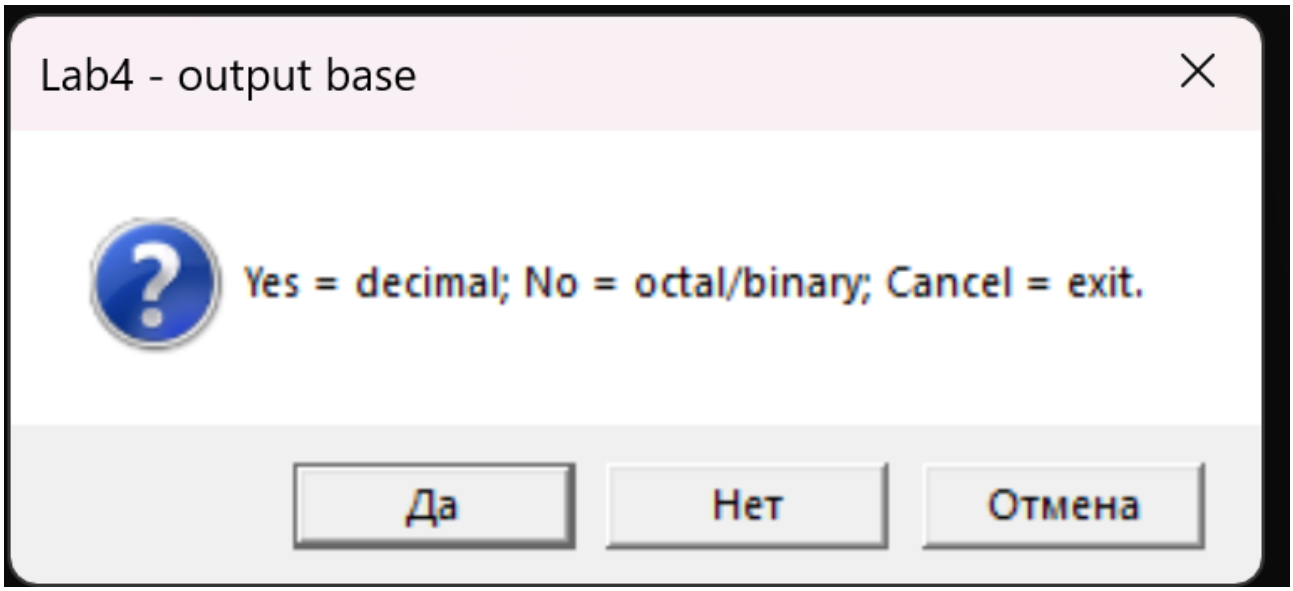


Рисунок 6 — Второй Popup: «Да» выбирает DEC, «Нет» ведёт к выбору OCT/BIN.

Требуемый вывод	Последовательность ответов
HEX, основание 16	Да в первом диалоге.
DEC, основание 10	Нет → Да.

Требуемый вывод	Последовательность ответов
OCT, основание 8	Нет → Нет → Да.
BIN, основание 2	Нет → Нет → Нет.

Popup вызывается с типом 3 + 32: 3 означает набор «Да/Нет/Отмена», 32 — значок вопроса. Возвращаемые коды: 6 — Да, 7 — Нет, 2 — Отмена. Тайм-аут 0 означает ожидание без ограничения времени. При отмене WScript.Quit(0) выполняется до создания нового выходного файла; старый text2.txt может остаться от предыдущего запуска.

7.3. Файловые операции и временные файлы

Фрагмент	Объяснение
new ActiveXObject("Scripting.FileSystemObject")	Создаёт объект доступа к файловой системе.
new ActiveXObject("WScript.Shell")	Создаёт объект запуска процессов и диалогов.
WScript.ScriptFullName	Полный путь текущего сценария; позволяет искать вход рядом с .js, независимо от текущей папки cmd.
GetParentFolderName(...)	Из полного имени сценария получает его каталог.
FileExists(input)	Проверяет наличие входного файла.
OpenTextFile(input, 1, false, 0)	Открывает существующий вход для чтения, в режиме системной ANSI-кодировки. В ASCII-примерах различия кодировок не проявляются.
reader.AtEndOfStream	Показывает, достигнут ли конец файла.
reader.ReadLine()	Читает одну строку, без символов перевода строки.
/^\s*$/.test(line)	Определяет пустую строку или строку только из пробельных символов.
CreateTextFile(output, true, true)	Создаёт выход с перезаписью и Unicode-кодировкой UTF-16LE.
WriteLine(...) / Close()	Записывает строку с переводом строки / закрывает поток.

bridge создаёт в системном временном каталоге два имени через GetTempName: запрос и ответ. GetSpecialFolder(2) обозначает временный каталог. Запрос содержит режим, левый операнд, операцию и правый операнд на отдельных строках. Функция quote заключает пути в двойные кавычки для корректной работы с пробелами. После выполнения запрос и прочитанный ответ удаляются через DeleteFile.

Пример запроса для 9 × 6: binary, 9, *, 6. Успешный ответ: первая строка OK, вторая — 54. При ошибке вместо числа поступает ERROR с пояснением; сценарий не должен выдавать заранее заданный ответ.

7.4. Запуск PowerShell-моста

```
powershell.exe -NoLogo -NoProfile -STA -ExecutionPolicy Bypass -File "...\calculator.ps1" -Request "..."
     -Response "..."
```

Параметр	Назначение
-NoLogo	Не выводить приветственный баннер PowerShell.
-NoProfile	Не загружать пользовательские профили, чтобы они не влияли на выполнение.
-STA	Выполнить в однопоточном COM-apartment, используемом интерфейсом буфера обмена. Это не означает запуск всего Windows в один поток.
-ExecutionPolicy Bypass	Задать политику для этого процесса без постоянной смены системных настроек. Не отменяет AppLocker, WDAC и групповые запреты.
-File	Указывает исполняемый файл PowerShell.
-Request / -Response	Именованные параметры нашего calculator.ps1, пути к файлам обмена.
shell.Run(command, 0, true)	Запустить мост со скрытым окном и дождаться завершения; получить код выхода.

Поздняя версия моста ищет процессы CalculatorApp, Calculator, calc, win32calc, активирует окно и вводит операнды. Для чтения результата пробует UI Automation по известным идентификаторам дисплея, затем Ctrl+C через WScript.Shell. Идентификаторы зависят от реализации интерфейса; наличие этого кода само по себе не гарантирует совместимость. Исходный Word-документ не содержит листинга calculator.ps1, поэтому точную его версию на момент каждого скриншота установить по документу нельзя.

7.5. Разбор записи чисел

Префиксы: 0x или 0X — основание 16, @ — основание 2, ведущий 0 перед другой цифрой — основание 8, без префикса — основание 10. Запись 0.5 считается десятичной, потому что после первого нуля стоит точка; 010.4 считается восьмеричной. Цифры проверяются: например, 08 и @102 недопустимы.

Число переводится во внутреннее десятичное значение по позиционной формуле. Для целой части используется n = n × base + digit; для дробной — сумма digit/base^k. Пример 0x1.A = 1 + 10/16 = 1.625; 0x2.8 = 2 + 8/16 = 2.5; сумма 4.125. Дробный разделитель входа всегда точка, независимо от региональных настроек.

parseInt(c, 16) в number() используется для получения значения одной цифры 0…F; затем её значение сравнивается с выбранным основанием. Это предотвращает частичное принятие недопустимого числа. Переменная prefix в данном листинге не участвует в возвращаемом значении; фактически для результата number() нужны base и value.

7.6. Парсер и приоритет операций

parse хранит позицию pos и пропускает пробелы через ws. atom обрабатывает число, унарный знак и выражение в скобках. term объединяет атомы операциями * и /. expr объединяет термы операциями + и −. Вложенный вызов expr при встрече скобок обеспечивает отдельный разбор подвыражения.

Для 2+3*4 строится узел + с левым ребёнком 2 и правым ребёнком — узлом *(3,4). Поэтому сначала вычисляется 12, затем 14. Для (2+3)*4 корнем становится *, сначала получается 5, затем 20. Циклы в term и expr обеспечивают левую ассоциативность:

8/4/2 = (8/4)/2 = 1, а не 4.

Дерево хранит объекты kind: number, unary или binary. evaluate рекурсивно получает значения детей. Для binary он вызывает переданный callback calculator(a, op, b), а callback — bridge. Для unary знак применяется в JScript; это не отдельное нажатие операции в Калькуляторе. Если бинарных вызовов не было, identity передаёт итоговое число в приложение для чтения.

Входные выражения не исполняются как программный код. В этой версии JScript парсер включён непосредственно в script3.js, поэтому eval для загрузки отдельного файла тоже не требуется. Строка module.exports относится к возможности тестирования ядра в Node.js; в WSH API помещается в root.LabEngine.

7.7. Форматирование и ограничения точности

format делит число на целую и дробную части. Целая часть преобразуется whole.toString(base), буквы A…F переводятся в верхний регистр. Дробная часть многократно умножается на основание; её целая часть даёт очередную цифру. После 12 цифр процесс останавливается, при ненулевом остатке добавляется [truncated]. Это усечение, а не округление до ближайшего.

Функция format сначала добавляет префикс результата, но строка записи через replace удаляет его, чтобы получить формат как в образце: например, 176 вместо 0x176. Заголовок Output base: 16 сообщает, в каком основании читать эти числа. Префиксы внутри исходного выражения слева сохраняются. Таким образом, основания входа и выхода — разные настройки.

Внутренние Number в JScript — IEEE-754 double. Для безопасных целых проверяется модуль не больше 9007199254740991; дроби могут иметь погрешности двоичного представления. Длина входного выражения ограничена 2000 символами, глубина atom — 64, число его вызовов — 500. Нельзя описывать эту реализацию как вычисления с произвольной точностью.

7.8. Ошибки и коды завершения

Ошибки строки перехватываются try/catch: записывается Line N: выражение -> ERROR: сообщение, затем программа продолжает следующие строки. Поэтому наличие text2.txt ещё не означает успех. Проверяются содержимое строк и число errors в финальном сообщении. При наличии хотя бы одной ошибки выполняется Quit(1), иначе Quit(0).

Примеры причин: недопустимая цифра; незакрытая скобка; лишний символ; деление на ноль; отсутствие calculator.ps1; отказ активации окна; отсутствие результата в буфере обмена. Если получено Calculator did not copy a result to the clipboard, это ошибка обмена с приложением, а не доказательство неверной записи выражения.

7.9. Подтверждённые результаты на снимках

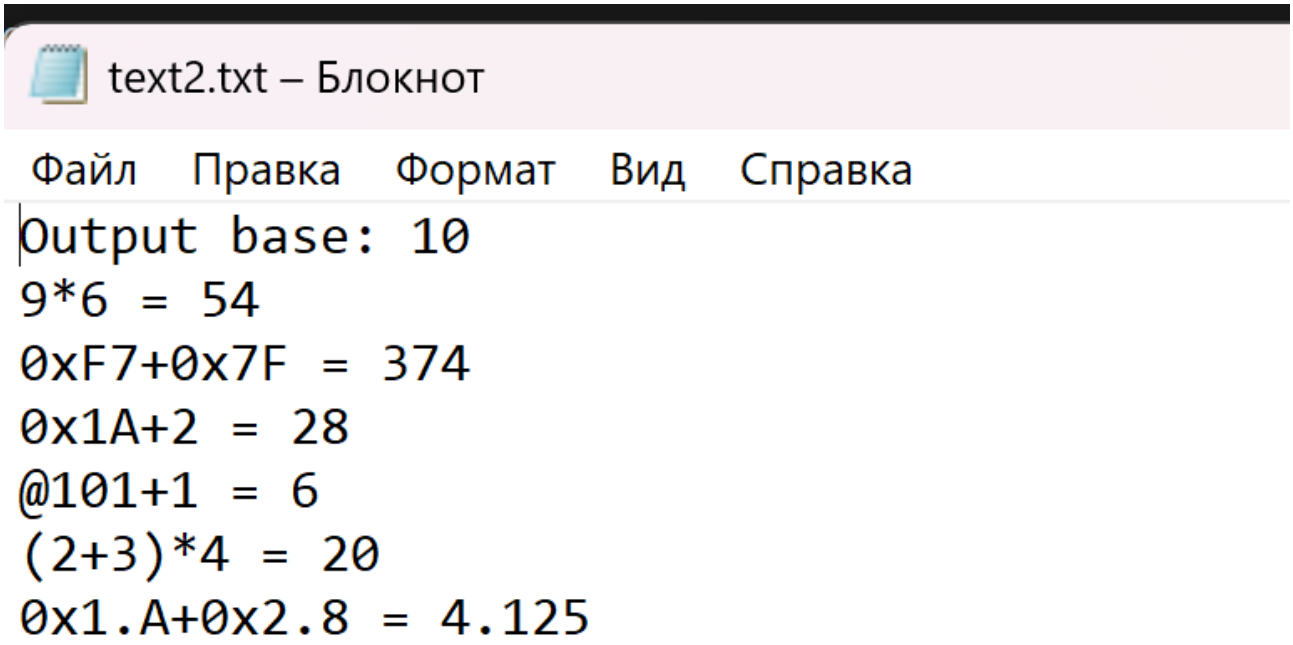


Рисунок 7 — Десятичный вывод шести исходных примеров. Все показанные числа совпадают с контрольными.

Выражение	DEC-результат	Проверка
9*6	54	Обычное умножение.
0xF7+0x7F	374	247 + 127.
0x1A+2	28	26 + 2.
@101+1	6	5 + 1.
(2+3)*4	20	Скобки выполняются первыми.
0x1.A+0x2.8	4.125	1.625 + 2.5.

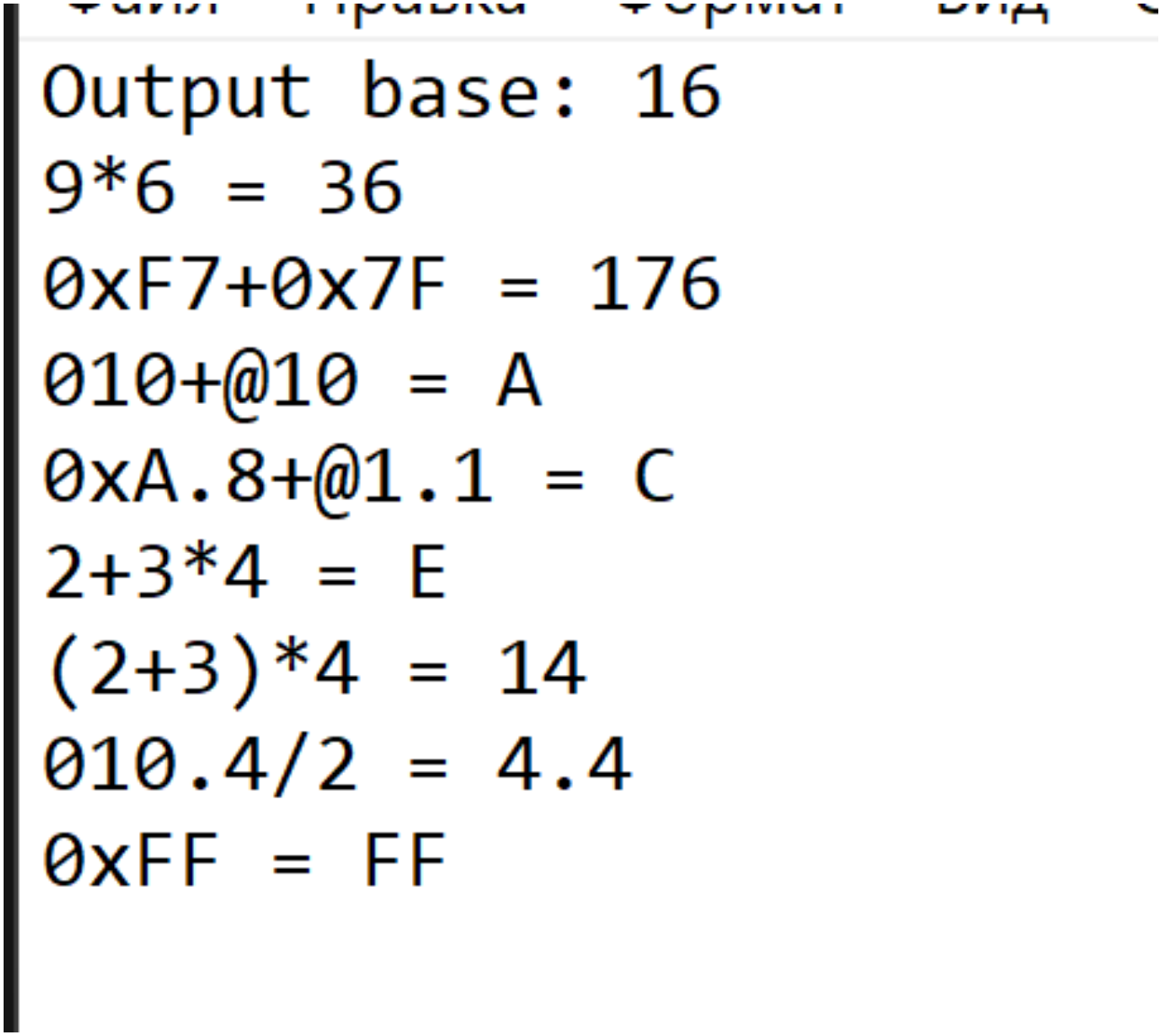


Рисунок 8 — HEX-вывод восьми дополнительных примеров: 36, 176, A, C, E, 14, 4.4 и FF.

Пример на снимке	Значение DEC	Запись HEX
9*6	54	36
0xF7+0x7F	374	176
010+@10	8 + 2 = 10	A
0xA.8+@1.1	10.5 + 1.5 = 12	C
2+3*4	14	E
(2+3)*4	20	14
010.4/2	8.5 / 2 = 4.25	4.4
0xFF	255	FF

Эти изображения подтверждают показанные DEC- и HEX-выходные файлы. Снимков OCT- и BIN-вывода в исходном документе нет. Последующие сообщения об ошибке буфера обмена в переписке показывают, что не каждый запуск или версия моста работали одинаково; успешные старые результаты не доказывают стабильность всех

последующих запусков.

Проверка OCT/BIN и ошибок рекомендуется отдельно, но исходный PDF не содержит прямого требования запускать программу ровно четыре раза. Он требует поддержку четырёх оснований и диалоговый выбор. Отсутствующие проверки нельзя обозначать как выполненные.

8. Итоги и оставшиеся замечания

Первое задание: на снимке получено и перенесено число 54; подпись отображена неполно. Второе: видны HEX 176 и DEC 374, Калькулятор в обычном режиме; поясняющие слова искажены. Третье: показаны корректные DEC-ответы для шести и HEX-ответы для восьми выражений; в них присутствуют дроби, скобки, приоритет операций и одиночное число.

Изучены объекты WScript, WScript.Shell и Scripting.FileSystemObject; запуск приложений, ожидание, активация окон, SendKeys, Popup, чтение и запись файлов. Реализован разбор выражений с учётом оснований и приоритетов. Практика показала, что арифметика и передача результата — отдельные этапы: верное число на дисплее не исключает ошибок фокуса, буфера обмена и кириллицы.

Для завершённого оформления первых двух заданий нужно приложить реальные повторные снимки с полными русскими подписями. Для заявления о проверке OCT/BIN нужны соответствующие фактические результаты. Эти замечания описывают наблюдаемые границы, а не подменяют их утверждением о полном успехе.

Приложение А. Снимки листингов первых двух сценариев

Сохранены исходные скриншоты кода. В текстовых приложениях Б и В приведены читаемые листинги той же версии с CalcKeys/NoteKeys. Это историческая версия до перехода к Unicode-файлам.

```
Файл  Правка  Формат  Вид  Справка
Option Explicit
Dim sh
Set sh = CreateObject("WScript.Shell")
If sh.Popup("Закройте другие окна Калькулятора и Блокнота. Переключите раскладку на английскую. Во время работы
sh.Run "calc.exe", 1, False
WScript.Sleep 2000
sh.Run "notepad.exe", 1, False
WScript.Sleep 2000
CalcKeys "%1"
CalcKeys "{ESC}"
CalcKeys "9*6="
CalcKeys "^c"
NoteKeys "Результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"

Sub ActivateCalc()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Калькулятор") Or sh.AppActivate("Calculator") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Калькулятор. Закройте лишние окна и повторите.", 16, "Лабораторная 4"
    WScript.Quit 1
End Sub
Sub ActivateNotepad()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Блокнот") Or sh.AppActivate("Notepad") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Блокнот. Закройте лишние окна и повторите.", 16, "Лабораторная 4"
    WScript.Quit 1
End Sub
```

Рисунок А.1 — Начало script1.vbs и процедуры ожидания окон.

```
    WScript.Quit 1
End Sub
Sub CalcKeys(keys)
    ActivateCalc
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
Sub NoteKeys(keys)
    ActivateNotepad
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
```

Рисунок А.2 — Окончание script1.vbs: CalcKeys и NoteKeys.

```
Option Explicit
Dim sh
Set sh = CreateObject("WScript.Shell")
If sh.Popup("Закройте другие окна Калькулятора и Блокнота. Переключите раскладку на английскую. Во время работ
sh.Run "calc.exe", 1, False
WScript.Sleep 2000
sh.Run "notepad.exe", 1, False
WScript.Sleep 2000
CalcKeys "%3"
CalcKeys "{F5}"
CalcKeys "{ESC}"
CalcKeys "F7{+}7F="
CalcKeys "^c"
NoteKeys "Шестнадцатеричный результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"
CalcKeys "{F6}"
CalcKeys "^c"
NoteKeys "Десятичный результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"
CalcKeys "%1"

Sub ActivateCalc()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Калькулятор") Or sh.AppActivate("Calculator") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Калькулятор. Закройте лишние окна и повторите.", 16, "Лабораторная 4"
    WScript.Quit 1
End Sub
```

Рисунок А.3 — Начало script2.vbs и переходы HEX/DEC.

```
End Sub
Sub ActivateNotepad()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Блокнот") Or sh.AppActivate("Notepad") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Блокнот. Закройте лишние окна и повторите
    WScript.Quit 1
End Sub
Sub CalcKeys(keys)
    ActivateCalc
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
Sub NoteKeys(keys)
    ActivateNotepad
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
```

Рисунок А.4 — Окончание script2.vbs, активация Блокнота и отправка клавиш.

Приложение Б. Листинг script1.vbs на скриншотах

```
Option Explicit
Dim sh
Set sh = CreateObject("WScript.Shell")
If sh.Popup("Закройте другие окна Калькулятора и Блокнота. Переключите раскладку на английскую. Во время
     работы не трогайте мышь и клавиатуру. Буфер обмена будет заменён. Продолжить?", 0, "Лабораторная
    4", 1 + 48) <> 1 Then WScript.Quit 0
sh.Run "calc.exe", 1, False
WScript.Sleep 2000
sh.Run "notepad.exe", 1, False
WScript.Sleep 2000
CalcKeys "%1"
CalcKeys "{ESC}"
CalcKeys "9*6="
CalcKeys "^c"
NoteKeys "Результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"

Sub ActivateCalc()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Калькулятор") Or sh.AppActivate("Calculator") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Калькулятор. Закройте лишние окна и повторите.", 16, "Лабораторная
    4"
    WScript.Quit 1
End Sub
Sub ActivateNotepad()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Блокнот") Or sh.AppActivate("Notepad") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Блокнот. Закройте лишние окна и повторите.", 16, "Лабораторная 4"
    WScript.Quit 1
End Sub
Sub CalcKeys(keys)
    ActivateCalc
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
Sub NoteKeys(keys)
    ActivateNotepad
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
```

Приложение В. Листинг script2.vbs на скриншотах

```
Option Explicit
Dim sh
Set sh = CreateObject("WScript.Shell")
If sh.Popup("Закройте другие окна Калькулятора и Блокнота. Переключите раскладку на английскую. Во время
     работы не трогайте мышь и клавиатуру. Буфер обмена будет заменён. Продолжить?", 0, "Лабораторная
    4", 1 + 48) <> 1 Then WScript.Quit 0
sh.Run "calc.exe", 1, False
WScript.Sleep 2000
sh.Run "notepad.exe", 1, False
WScript.Sleep 2000
CalcKeys "%3"
CalcKeys "{F5}"
CalcKeys "{ESC}"
CalcKeys "F7{+}7F="
CalcKeys "^c"
NoteKeys "Шестнадцатеричный результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"
CalcKeys "{F6}"
CalcKeys "^c"
NoteKeys "Десятичный результат: "
NoteKeys "^v"
NoteKeys "{ENTER}"
CalcKeys "%1"

Sub ActivateCalc()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Калькулятор") Or sh.AppActivate("Calculator") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Калькулятор. Закройте лишние окна и повторите.", 16, "Лабораторная
    4"
    WScript.Quit 1
End Sub
Sub ActivateNotepad()
    Dim i
    For i = 1 To 30
        If sh.AppActivate("Блокнот") Or sh.AppActivate("Notepad") Then
            WScript.Sleep 500
            Exit Sub
        End If
        WScript.Sleep 500
    Next
    MsgBox "Не удалось активировать Блокнот. Закройте лишние окна и повторите.", 16, "Лабораторная 4"
    WScript.Quit 1
End Sub
Sub CalcKeys(keys)
    ActivateCalc
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
Sub NoteKeys(keys)
    ActivateNotepad
    sh.SendKeys keys
    WScript.Sleep 700
End Sub
```

Приложение Г. Листинг script3.js из исходного Word-документа

Листинг сохранён без изменения алгоритма; визуальный перенос длинных строк в документе не является изменением исходного кода.

```
// Windows Script Host JScript (run with cscript.exe //nologo script3.js).
var fso = new ActiveXObject("Scripting.FileSystemObject");
var shell = new ActiveXObject("WScript.Shell");
var dir = fso.GetParentFolderName(WScript.ScriptFullName);
/* ES3: Windows Script Host JScript + Node.js test runner. No eval(). */
```

```
(function (root) {
    function fail(s) { throw new Error(s); }
    function parse(source) {
        var pos = 0, count = 0;
        if (source.length > 2000) fail("Expression exceeds 2000 characters");
        function ws() { while (/\s/.test(source.charAt(pos)) && pos < source.length) pos++; }
        function number() {
            ws(); var start = pos, base = 10, prefix = "";
            if (source.substr(pos, 2).toLowerCase() === "0x") { base = 16; pos += 2; prefix = "0x"; }
            else if (source.charAt(pos) === "@") { base = 2; pos++; prefix = "@"; }
            else if (source.charAt(pos) === "0" && /[0-9]/.test(source.charAt(pos + 1)) && pos + 1 <
    source.length) base = 8;
            var digits = "", dots = 0;
            while (pos < source.length && /[0-9a-fA-F.]/.test(source.charAt(pos))) {
                var c = source.charAt(pos++); if (c === ".") dots++;
                else if (parseInt(c, 16) >= base) fail("Invalid digit for base " + base + " at " + pos);
                digits += c;
            }
            if (!digits || digits === "." || dots > 1) fail("Invalid number at " + (start + 1));
            var parts = digits.split("."), n = 0, i, scale = base;
            for (i = 0; i < parts[0].length; i++) n = n * base + parseInt(parts[0].charAt(i), 16);
            if (parts.length > 1) for (i = 0; i < parts[1].length; i++) { n +=
    parseInt(parts[1].charAt(i), 16) / scale; scale *= base; }
            if (!isFinite(n) || Math.abs(n) > 9007199254740991) fail("Number exceeds safe numeric
    range");
            return {kind: "number", value: n};
        }
        function atom(depth) {
            if (depth > 64 || ++count > 500) fail("Expression too complex");
            ws(); var c = source.charAt(pos), a;
            if (c === "+" || c === "-") { pos++; return {kind: "unary", op: c, child: atom(depth + 1)};
    }
            if (c === "(") { pos++; a = expr(depth + 1); ws(); if (source.charAt(pos++) !== ")")
    fail("Missing closing parenthesis"); return a; }
            return number();
        }
        function term(depth) { var a = atom(depth), c; ws(); while ((c = source.charAt(pos)) === "*" ||
    c === "/") { pos++; a = {kind:"binary", op:c, left:a, right:atom(depth)}; ws(); } return a; }
        function expr(depth) { var a = term(depth), c; ws(); while ((c = source.charAt(pos)) === "+" ||
    c === "-") { pos++; a = {kind:"binary", op:c, left:a, right:term(depth)}; ws(); } return a; }
        var tree = expr(0); ws(); if (pos !== source.length) fail("Unexpected character at " + (pos +
    1)); return tree;
    }
    function evaluate(tree, calculator) {
        if (tree.kind === "number") return tree.value;
        if (tree.kind === "unary") return (tree.op === "-" ? -1 : 1) * evaluate(tree.child, calculator);
        var a = evaluate(tree.left, calculator), b = evaluate(tree.right, calculator);
        if (tree.op === "/" && b === 0) fail("Division by zero");
        var n = calculator(a, tree.op, b);
        if (!isFinite(n) || Math.abs(n) > 9007199254740991) fail("Result exceeds safe numeric range");
        return n;
    }
    function format(n, base) {
        if (base !== 2 && base !== 8 && base !== 10 && base !== 16) fail("Unsupported base");
        if (!isFinite(n) || Math.abs(n) > 9007199254740991) fail("Result exceeds safe numeric range");
        var sign = n < 0 ? "-" : "", x = Math.abs(n), whole = Math.floor(x), fraction = x - whole;
        var text = whole.toString(base).toUpperCase(), tail = "", i, d;
        for (i = 0; i < 12 && fraction > 0; i++) { fraction *= base; d = Math.floor(fraction); tail +=
    d.toString(base).toUpperCase(); fraction -= d; }
        tail = tail.replace(/0+$/, "");
        return sign + (base === 16 ? "0x" : base === 8 ? "0" : base === 2 ? "@" : "") + text + (tail ?
    "." + tail : "") + (fraction > 0 ? " [truncated]" : "");
    }
    var api = {parse:parse, evaluate:evaluate, format:format};
    if (typeof module !== "undefined" && module.exports) module.exports = api; else root.LabEngine =
    api;
}(this));

function quote(s) { return '"' + s + '"'; }
function bridge(mode, a, op, b) {
    var req = fso.BuildPath(fso.GetSpecialFolder(2), fso.GetTempName());
    var res = fso.BuildPath(fso.GetSpecialFolder(2), fso.GetTempName());
    var file = fso.CreateTextFile(req, true, true);
    file.WriteLine(mode); file.WriteLine(String(a)); file.WriteLine(op); file.WriteLine(String(b));
    file.Close();
    var code = shell.Run('powershell.exe -NoLogo -NoProfile -STA -ExecutionPolicy Bypass -File ' +
    quote(dir + '\\calculator.ps1') + ' -Request ' + quote(req) + ' -Response ' + quote(res), 0, true);
    fso.DeleteFile(req);
```

```
    if (!fso.FileExists(res)) throw new Error('PowerShell bridge failed to start');
    file = fso.OpenTextFile(res, 1, false, -1); var text = file.ReadAll(); file.Close();
    fso.DeleteFile(res);
    var lines = text.split('\r\n');
    if (code !== 0 || lines[0] !== 'OK') throw new Error(text);
    var value = Number(lines[1]); if (!isFinite(value)) throw new Error('Invalid Calculator response');
    return value;
}
var choice = shell.Popup('Output base: Yes = hexadecimal, No = another base, Cancel = exit.', 0, 'Lab4 -
     output base', 3 + 32);
if (choice === 2) WScript.Quit(0);
var base = 16;
if (choice === 7) {
    choice = shell.Popup('Yes = decimal; No = octal/binary; Cancel = exit.', 0, 'Lab4 - output base', 3
    + 32);
    if (choice === 2) WScript.Quit(0);
    if (choice === 6) base = 10;
    else {
        choice = shell.Popup('Yes = octal; No = binary; Cancel = exit.', 0, 'Lab4 - output base', 3 +
    32);
        if (choice === 2) WScript.Quit(0);
        base = choice === 6 ? 8 : 2;
    }
}
var input = dir + '\\text1.txt', output = dir + '\\text2.txt';
if (!fso.FileExists(input)) { WScript.Echo('Missing text1.txt'); WScript.Quit(1); }
// Input is ASCII (expressions only), output UTF-16LE for Russian Windows compatibility.
var reader = fso.OpenTextFile(input, 1, false, 0), results = [], lineNumber = 0, failures = 0;
while (!reader.AtEndOfStream) {
    var line = reader.ReadLine(); lineNumber++;
    if (/^\s*$/.test(line)) continue;
    try {
        var tree = LabEngine.parse(line), calls = 0;
        var result = LabEngine.evaluate(tree, function(a, op, b) { calls++; return bridge('binary', a,
    op, b); });
        if (!calls) result = bridge('identity', result, '', 0);
        results.push(line + ' = ' + LabEngine.format(result, base).replace(/^(-?)(0x|@)/,
    '$1').replace(/^(-?)0(?=[0-7])/, '$1'));
    } catch (e) { failures++; results.push('Line ' + lineNumber + ': ' + line + ' -> ERROR: ' +
    e.message); }
}
reader.Close();
var writer = fso.CreateTextFile(output, true, true);
writer.WriteLine('Output base: ' + base);
for (var i = 0; i < results.length; i++) writer.WriteLine(results[i]);
writer.Close();
WScript.Echo('Written: ' + output + '; expressions: ' + results.length + '; errors: ' + failures);
WScript.Quit(failures ? 1 : 0);
```